

DKSI Tools — Revit 2027 Add-in Technical Report

Field	Detail
Product	DKSI Tools — Revit 2027 Add-in Suite
Report Date	12 August 2026
Reporting Period	6 August 2026 – 12 August 2026
Target Platform	Autodesk Revit 2027 (.NET 10, net10.0-windows, x64)
Build	1.0.26224
Prepared for	Senior Management, Engineering Leads, and Programme Stakeholders
Prepared by	John Immanuel Serdan
Classification	Internal — Technical Status

1. Executive Summary

The add-in suite has moved from a working codebase to a **packaged, deployable product**. Three things changed materially in this period.

The toolset is finalised and surfaced. All active commands now appear as individual ribbon buttons across four labelled panels, replacing the single pull-down of the previous revision. Ten commands are on the ribbon.

The repository has been cleaned. 185 tracked files reduced to 153, removing a superseded installer system, stray test-run artefacts, and five commands that were neither on the ribbon nor in the brief — together with the engines and icons they exclusively owned. Nothing load-bearing was removed, and the build remains clean at zero warnings.

Deployment is solved, in two forms. A per-user installer requiring no administrator rights, and a machine-wide bundle for IT mass deployment. The per-user package now carries the paint takeoff assembly internally, so the full ten-button ribbon is available on a locked-down workstation with no admin involvement at all.

Three items require leadership attention rather than engineering:

Code signing is unresolved and is the principal deployment risk. Both installers are unsigned. SmartScreen warns on every workstation, and corporate policy may block unsigned executables outright. An organisational code-signing certificate resolves this *and* removes the separate certificate-trust step now required on each machine. This is a purchasing decision, not a technical one.

The finish-calculation optimisation is not yet validated against a production model. A quadratic performance defect was corrected in the core measurement engine this period. The change is behaviour-preserving by construction and its edge cases are individually documented, but the solution carries **no automated tests**, and the engine's output is areas that appear on issued drawings. A comparison harness has been supplied; validation is a prerequisite to office-wide release.

Engineering capacity is constrained by AI tier limits, not by headcount. The suite in this report was built and packaged by a single engineer working with AI assistance. Standard-tier usage caps are routinely exhausted mid-task, halting active development and idling paid engineering hours. Section 7 sets out the business case for high-tier allocation, and the cost comparison against hiring a dedicated development team to achieve the same output.

Headline position: the product is feature-complete and packaged. Two gates remain before office-wide rollout — numerical validation of the finish engine, and a decision on code signing. A third decision, on engineering capacity, governs the rate at which the remaining pipeline in Section 6 can be delivered.

2. Finalised Toolset

Ten commands are present in build 1.0.26224, grouped into four ribbon panels on the **DKSI** tab — *Model* (items 1–3), *Finishes & Paint* (items 4–8), *Reporting* (item 9) and *Time* (item 10).

#	Capability	Status	Disposition
1	Door Lining & Door Material — lining clash resolution and material write	Complete	Ready for Senior Project Engineer testing
2	Door Udvendig — exterior placeholder room substitution	Complete	Ready for Senior Project Engineer testing
3	Place Skirting (Wall Sweep) — native wall-sweep placement	80%	Cleared for production release
4	Adjust Room Boundaries — automated spatial adjustment	Complete	Ready for Senior Project Engineer testing
5	Painted Surface Area — room-bounded paint takeoff	Complete	Ready for Senior Project Engineer testing
6	Painted Area (project wide) — element-centric paint area	Complete	Ready for Senior Project Engineer testing
7	Show / Hide Paint Areas — calculation geometry visibility	Complete	Ready for Senior Project Engineer testing
8	Surface Schedules — build/repair the three takeoff views	Complete	Ready for Senior Project Engineer testing
9	Export Schedules — built-in Excel exporter	Complete	Ready for Senior Project Engineer testing
10	Time Management & Monitoring	30% production	In progressive rollout

2.1 Ribbon architecture — flattened from a pull-down

The previous revision collapsed the tab into a single "DKSI Tools" pull-down. That has been reversed. The original argument — that nobody scans nine icons — is sound but outweighed: these are tools run *while working*, and one additional click on every use costs more than a wide tab.

Panels now carry the grouping that separators previously carried inside the menu. Because Revit labels each panel, the grouping is more visible than before, not less. Button labels are two-line, because a panel button balances its text under the icon and truncates a long name otherwise.

A secondary benefit: the pull-down created an availability trap. A `PulldownButton` carrying an `AvailabilityClassName` greys out the parent and takes every child with it, so the parent deliberately carried none — otherwise Time Tracking, which must work with no document open, was unreachable. With each command as its own top-level button, each carries its own availability and the trap is gone.

2.2 Paint tooling — provenance

Items 5, 6 and 7 are commands in the **Painted Material Takeoff** assembly, a separate product, not in the DKSI assembly. They are surfaced on the DKSI tab so the office learns one ribbon rather than two, and that product no longer builds a tab of its own.

Two engineering notes carry forward:

- A `PushButton` may name any assembly on disk, so these commands need not be copied into this codebase — which matters, as no source tree exists for that product.
- **An availability class must resolve from the button's own assembly.** Revit looks for `AvailabilityClassName` inside the assembly the button names, not inside the add-in that built the ribbon. Supplying a DKSI availability type produced a load failure at every start-up which named DKSI as the culprit. The three buttons now use that product's own `DocumentAvailability`.

If the paint takeoff assembly is absent, these three buttons omit themselves and the ribbon shows seven. That is deliberate: a button that throws "file not found" on click is worse than a shorter panel.

2.3 Commands retained but not surfaced

Three DKSI commands remain in the codebase, fully functional, deliberately off the ribbon:

Command	Sole capability it provides
<code>FinishSurfaceAreaCommand</code>	Writes Wall/Floor/Ceiling Finish Area, Wall Paint Area and Net Floor Area to rooms and elements, and exports the per-room per-material CSV
<code>PaintTakeoffCommand</code>	Builds DKSI Paint Takeoff by Room — the only route to regenerating that schedule after model change
<code>PaintHighlightCommand</code>	Draws the measured paint area of a selected element, showing <i>which</i> surface a takeoff row measured

These are not equivalent to items 5–7 above. Painted Surface Area is a takeoff, not a parameter write, and does not substitute for Finish Surface Area. Restoring any of the three is a single `AddButton` call.

Note that the underlying engines run regardless: `FinishAutomation` drives the finish pass off `DocumentChanged`, and `PaintHighlightService` is registered at start-up whether or not its button exists. Room and element parameters continue to update automatically.

3. Parameter Linking — Paused

Status: paused pending finalisation of the data mapping list.

Parameter Linking (Text-to-Type to Instance binding) was reported at 100% in the previous cycle. It has been withdrawn from the active capability table at the programme owner's direction and is **not** included in build 1.0.26224 as a surfaced command.

This is a scope hold, not a defect or a regression. No linking code has been deleted; the pause is on surfacing and on the mapping specification, not on the implementation.

Gating item: the complete list of source-to-target parameter mappings. Until that list is finalised, any binding shipped would encode an incomplete mapping into production models — and parameter bindings, once written across a model, are materially harder to correct than to defer.

Re-entry: on delivery of the finalised mapping list, the work required is specification of the mapping table and re-surfacing of the command. It should be re-baselined at that point rather than resumed at its previous 100%, since the percentage described a scope that is now being redefined.

4. Current Architecture Status

4.1 Repository cleanup

Measure	Before	After
Tracked files	185	153
C# source files (add-in)	72	67
C# lines	~21,500	19,769
Build warnings	0	0

Removed — superseded infrastructure

Item	Justification
Installer/ (root) — 6 files	A ZIP-and-copy install system predating the WiX packages. Verified unreferenced across every .ps1, .wxs, .cs, .md and .txt in the tree. Two live systems replaced it
.csv / .log run artefacts — 8 files	Output from test runs of the Dynamo scripts, committed in error

Removed — inactive commands and their exclusive dependencies

Command	Also removed
About	—
Stamp Review Date	—
Set Up Finish Schedules	Schedules/CeilingTakeoffBuilder.cs
Sync Material Parameters	Materials/ (3 files)
Diagnose Parameters	Diagnostics/ParameterReport.cs, UI/DiagnoseParamsWindow

Plus eight embedded icons orphaned by those removals. Each deletion was verified by reference: nothing outside the deleted set mentioned any of them in code.

Retained — load-bearing despite appearing inactive

Three items look removable and are not:

- **RoomFinishCalculator** — FinishAutomation drives it off DocumentChanged, registered at start-up. Removing it breaks Revit launch, not merely a button.
- **PaintTakeoffBuilder** — used by SurfaceScheduleBuilder, and Surface Schedules **is** on the ribbon.
- **Overlay/** — PaintHighlightService is registered at start-up regardless of its button.

Retained — deliberate archive

- parked/ — code deleted before this repository existed; it survives nowhere else
- Dynamo .dyn / .py graphs — the closest thing to a written specification this programme has, as recorded in .gitignore

4.2 Directory structure

```
Computational Design Assistant/
├── Door From-To Rooms/      Dynamo source (specification) + README
├── Door Window Linings/    Dynamo source (specification) + README
├── Export Excel/           Dynamo source
├── Finish Surface Area/    Dynamo source + reference notes
├── Material Browser to .../ Dynamo source
├── DKSI_Automation_Status_Report.{md,docx,pdf}
├── DKSI_Revit_Addin_Technical_Report.md ← this document
├── Revit Add-in/
│   ├── Directory.Build.props single source of truth for the Revit version
│   ├── docs/               trial guide, session summaries
│   └── installer/
│       ├── DKSI-Revit-Tools.wxs      per-user MSI
│       ├── DKSI-Revit-Tools-AllUsers.wxs machine-wide MSI
│       ├── DKSI-Suite.wxs            bundle chaining both products
│       ├── DKSI-Revit-Tools.iss      Inno Setup, per-user
│       ├── PaintTakeoff/             repackaging + trust assets
│       └── retired-2026-08, smb-checklist
├── parked/
├── tools/
│   └── build, deploy, diagnostic and uninstall scripts
├── src/Cda.Revit.Addin/
│   ├── Automation/          OpeningAutomation
│   ├── Commands/           10 command classes
│   ├── Doors/ Linings/     Udvendig resolution, lining clash resolution
│   ├── Excel/              XlsxWriter – no third-party dependency
│   ├── Finishes/           measurement engine, automation, staleness updater
│   ├── Infrastructure/      Log, CommandBase, Icons, Availability, task queue
│   ├── Overlay/            PaintHighlight (registered at start-up)
│   ├── Resources/Icons/    16 PNGs, all referenced
│   ├── Schedules/          PaintTakeoff, SurfaceSchedule*, ScheduleExporter
│   ├── Sweeps/             skirting generation and placement
│   ├── TimeTracking/       UI/
│   └── CdaApplication.cs, RibbonBuilder.cs, manifest, project file
```

4.3 Dependency posture

- **Zero NuGet package references.** Nothing third-party ships with the product; the Excel exporter writes .xlsx directly.
- **Revit API assemblies referenced, never copied** (Private=false). Shipping a private copy causes type-identity failures at runtime.

- **.NET 10 supplied by the host.** Revit 2027 runs on .NET 10 and installs it as its own prerequisite. A Revit add-in is loaded into Revit.exe and therefore cannot bring its own runtime; a self-contained publish is not applicable to an in-process plug-in.

4.4 Engineering changes this period

Ten commits. Two are worth flagging technically.

Finish deduction indexing — performance. DoorWindowDeduction and CaseworkDeduction ran once per room and scanned every opening and casework instance in the model, calling `get_FromRoom`, `get_ToRoom` and `get_Room` on each. Those are spatial lookups, not property reads, and their results do not depend on which room is being measured. On a model with 400 rooms and 1,200 openings this performed **960,000 lookups to obtain what 1,200 would have provided** — quadratic in the two quantities that grow with project size. The deductions are now indexed by room, built once per phase.

Duplicate-install detection — supportability. The per-user and machine-wide packages carry the same add-in identifier. Revit does not support reading one add-in from two manifests: it loads one, ignores the other, and does not report which. A user who installed the product themselves and later received an IT rollout can therefore run months-old code with the current version on disk beside it — and every symptom of that presents as "the update did not work". The add-in now detects this at start-up and names the folder the session actually loaded from. This check cannot live in the installer: a machine-wide MSI deployed by SCCM or Intune runs as SYSTEM and would inspect the wrong profile.

5. Deployment & Packaging

Package	Scope	Rights	Contents	Use
DKSI-Revit-Tools-Setup-1.0.26224.exe	Per-user, %AppData%	None	All 10 commands, paint assembly included	Individual workstations
DKSI-Revit-Suite-1.0.26224.exe	Machine-wide, Program Files	Administrator	Chains both products	IT mass deployment
DKSI-Revit-Tools-AllUsers-...msi	Machine-wide	Administrator	Embedded in the bundle	Not distributed separately

Per-user and mass deployment are mutually exclusive. %AppData% resolves against the account running setup. A silent push from SCCM or Intune runs as SYSTEM and writes the add-in into SYSTEM's profile — setup reports success, every file is written, and no user's Revit ever sees it. The per-user package must be deployed in user context; machine-wide deployment requires the Program Files package.

Revit 2027 relocated the all-users add-in folder from C:\ProgramData\Autodesk\Revit\Addins\2027 to C:\Program Files\Autodesk\Revit\Addins\2027, and refuses manifests found in the former — silently, with one journal line. This was discovered when a packaged product installed perfectly and never loaded. All machine-wide packaging now targets the correct location.

5.1 Supporting tooling

Script	Purpose
Get-InstallDiagnostics.ps1	Read-only report explaining why a workstation shows no ribbon
Remove-PerUserInstall.ps1	Clears a per-user copy shadowing a machine-wide install
Uninstall-DKSI.ps1	Removes residue the bundle's own uninstall cannot reach
Compare-FinishCsv.ps1	Diffes before/after finish exports to validate the engine optimisation

6. Ongoing Scripting & Development Pipeline

Item	Status	Constraint
Automated-Void for Fitting Casework	Specified — not started	Awaiting capacity; specification below
Stair Void Automation	Not started	Capacity / sequencing
Auto-Generated Walls from Images	20% built — on hold	External vision API token limits and cost
Computer Vision Identification	Not started — deferred	Token / vision overhead
Danish-to-English Document Translator	Planned	Scope definition pending

6.1 Automated-Void for Fitting Casework — New

Objective. Automatically create and maintain void cuts in host walls where fitting casework is placed, removing the manual per-instance void work currently required to make casework read correctly in both geometry and finish quantities.

Why this is well-positioned. The measurement half of this feature already exists and is in production. The finish engine reads void cuts today in two places, using `InstanceVoidCutUtils.GetElementsBeingCut` to determine whether a casework void has already been carved out of a measured wall face — and to avoid double-deducting when it has. The skirting generator likewise breaks runs at casework. **The engine already understands casework voids; what is missing is the code that creates them.** This is an unusually low-risk addition: it writes into a model the rest of the suite already interprets correctly.

Functional specification

1. **Selection** — whole model by default, or current selection, matching the established pattern of the lining and skirting commands.
2. **Candidate identification** — casework instances that are wall-hosted, or that geometrically intersect a wall without an existing cut. Instances already cutting their host are skipped, so the command is safe to re-run.
3. **Void creation** — `InstanceVoidCutUtils.AddInstanceVoidCut` against the resolved host. Families lacking a void form are reported rather than silently ignored; the report names the family so it can be corrected once at the library level rather than per instance.

4. **Dry run** — a preview pass listing what would be cut, before anything is written. Consistent with Door Lining & Door Material, and the only responsible default for a command that alters host geometry across a model.
5. **Single undo** — one transaction for the whole run.
6. **Idempotence** — re-running changes nothing on instances already correct.

Integration plan

Stage	Work	Dependency
1	Sweeps/ or new Voids/ namespace — resolver, candidate query, reporting model	None
2	Command class + ribbon entry in the Model panel, beside Place Skirting	Stage 1
3	Staleness integration — flag affected rooms so finish areas recompute	FinishStaleUpdater already watches OST_Casework
4	Validation against a production model, comparing finish CSVs before and after	Compare-FinishCsv.ps1

Known risks

- **Family library dependency.** A casework family without a void form cannot cut its host. Expect a first-run report identifying families requiring library correction; this is a modelling-standards outcome, not a defect.
- **Host resolution ambiguity.** Casework placed against, rather than hosted in, a wall requires geometric resolution. The finish engine's existing fallback — testing GetElementsBeingCut and bounding-box proximity — provides a proven starting point.
- **Interaction with skirting.** Skirting already breaks at casework. Introducing real voids changes the geometry skirting is measured against, so stages 3 and 4 must be validated together rather than separately.

6.2 Stair Void Automation

Not commenced. Held in the backlog pending capacity. Shares the void-cutting infrastructure proposed in 6.1 and should be sequenced immediately after it, to reuse rather than duplicate that work.

6.3 Auto-Generated Walls from Images — *Primary Bottleneck*

Unchanged. Approximately 20% built and suspended. The blocker remains external: vision-API token limits and per-call cost. **Decision required from leadership** — approve a budget envelope, authorise investigation of a locally hosted model, or formally defer.

6.4 Computer Vision Identification

Deferred, dependent on 6.3. Actionable only once the vision-cost decision is made.

6.5 Danish-to-English Document Translator

Planned. Lower technical risk and not subject to the vision constraint; a candidate for opportunistic scheduling if the vision decision is delayed.

7. Engineering Capacity & Tooling Investment

As the pipeline expands to include advanced computational tools and complex automation projects, continued reliance on standard AI tiers creates severe productivity bottlenecks. Hitting constant usage limits on standard Pro accounts forces skilled engineering talent to wait idly, stalling production and delaying deliverables.

This section sets out the business case for investing in high-tier Claude allocations (Max ×5 / ×20), and the financial case for AI-assisted development against hiring a dedicated software development team to scale the proprietary add-ins described in this report.

7.1 The Bottleneck — why standard limits threaten project timelines

Standard AI developer tiers enforce strict usage caps. For a BIM Computational Engineer managing heavy algorithmic workflows, C# APIs and Python-driven Revit automation, these caps are routinely exhausted mid-task.

The cost of waiting. When an engineer encounters a hard limit, active development halts. Multi-hour build processes, debugging sessions and complex script generation are abruptly frozen.

Lost production time. Forced downtime translates directly into delayed milestone deliveries, slower project execution, and compromised agility against competing firms.

The solution. Claude Max ×5 or ×20 guarantees uninterrupted, high-capacity execution, ensuring engineering talent spends paid hours actively producing rather than waiting for capacity resets.

7.2 What is being requested — the plans and their cost

Plan	Price	Usage vs Pro	Status
Pro	USD 20/month, or USD 17/month billed annually (USD 200 up front)	Baseline	Current tier — the source of the bottleneck
Max ×5	USD 100/month ≈ PHP 5,800/month, PHP 69,600/year	5× more usage per session	Proposed minimum
Max ×20	USD 200/month ≈ PHP 11,600/month, PHP 139,200/year	20× more usage per session	Recommended

The request is one Max ×20 seat for the BIM Computational Engineer: USD 200 per month, approximately PHP 139,200 per year.

Because the engineer already holds a Pro subscription, the **incremental** cost is **USD 180 per month** — approximately PHP 10,440 per month, or **PHP 125,280 per year**.

Why ×20 rather than ×5. The bottleneck in Section 7.1 is *session* capacity, and the ×5 tier raises it fivefold where ×20 raises it twentyfold, for one additional USD 100 per month. Against a developer-team alternative starting at PHP 1.8 million, the difference between the two tiers — PHP 69,600 per year — is not a material saving, while the difference in headroom is the entire point of the request. If a lower commitment is preferred for a first term, ×5 is a valid starting point and can be upgraded without re-procurement.

7.3 Financial comparison — AI subscription vs. dedicated development team

To maintain, scale and extend complex Revit add-ins internally, leadership faces a clear financial fork. The table contrasts scaling via AI infrastructure against traditional hiring in the Philippine market.

Metric / Resource	Traditional Full Developer Team (Mid–Senior)	AI-Augmented Workflow (Claude Max ×5 / ×20 + existing add-ins)
Headcount required	2–3 developers + 1 QA engineer	1 existing BIM Computational Engineer + advanced AI
Annual salary & benefits	PHP 1,800,000 – PHP 3,000,000+ per year	Minimal — existing headcount cost
Software / infrastructure cost	High — multiple IDE licences, hardware, overhead	Claude Max ×5: USD 100/month. Max ×20: USD 200/month. ≈ PHP 69,600 / PHP 139,200 per year
Ramp-up / onboarding time	2–3 months: hiring, training, domain alignment	Immediate — leverages already-built add-ins
Output speed	Limited by human typing, meetings and coordination	Accelerated ~5× via automated code generation and debugging

7.4 Estimated financial savings

By empowering the current engineer with high-tier AI capability rather than onboarding a traditional software development team, the company saves an estimated **85–90% in operational and payroll expense**, preserving hundreds of thousands of pesos monthly while accelerating project delivery.

The published subscription pricing puts a firm figure on one side of that comparison:

	Annual cost	As a share of the developer-team cost
Claude Max ×5	USD 1,200 ≈ PHP 69,600	2.3% – 3.9%
Claude Max ×20	USD 2,400 ≈ PHP 139,200	4.6% – 7.7%
Traditional developer team	PHP 1,800,000 – 3,000,000+	100%

On these figures the incremental cost of the AI-augmented path is **roughly 5–8% of the traditional path at the ×20 tier** — a reduction of about 92–95%. The 85–90% estimate above is therefore **conservative**, and is retained as the headline figure on that basis.

*Basis and caveats: subscription pricing is Anthropic's published rate for Claude Max ×5 (USD 100/month) and ×20 (USD 200/month); prices exclude applicable tax and are subject to change. Peso figures are converted at approximately PHP 58 per USD — **confirm the prevailing rate at submission**. The developer-team range is a Philippine market estimate for mid–senior developers at the stated headcount, not a quotation. Existing engineer headcount cost is excluded from both columns, since it is unchanged either way.*

7.5 Maximising the existing add-in ecosystem

The organisation is not starting from scratch. It already owns functional, proprietary Revit add-ins built in-house — the suite documented in Sections 2 through 5 of this report: **19,769 lines of C# across 67 source files, packaged in three installer formats, with zero third-party dependencies.**

A newly hired development team would spend substantial time simply auditing, understanding and refactoring that existing codebase before producing anything new. The architectural detail in Section 4 illustrates the point directly: several components appear inactive and are in fact load-bearing, and a team without that context would either break them or spend weeks establishing it.

With high-capacity AI access, the current engineer can feed, optimise, scale and deploy the existing add-ins directly — converting prototype tools into enterprise-grade production assets without that ramp-up cost.

7.6 Recommendation

Approve one Claude Max ×20 subscription for the BIM Computational Engineer.

The ask	
Plan	Claude Max ×20 (20× Pro session capacity)
Seats	1
List price	USD 200 per month — approximately PHP 11,600 per month
Annual	USD 2,400 — approximately PHP 139,200
Incremental over the current Pro seat	USD 180 per month — approximately PHP 125,280 per year
Alternative being displaced	2–3 developers + 1 QA engineer, PHP 1,800,000 – 3,000,000+ per year
Procurement	Card or invoice, self-serve. No tender, no onboarding, no headcount change

This is a low-cost, high-yield investment that eliminates production bottlenecks, protects project timelines, and secures maximum return from existing software assets without expanding payroll. It is the lowest-expenditure decision in this report and among the highest by effect on delivery rate.

If a smaller first commitment is preferred, **Max ×5 at USD 100 per month (≈ PHP 69,600 per year)** delivers five times current capacity and can be upgraded to ×20 at any point without re-procurement.

8. Risks & Actions

#	Risk	Impact	Action	Owner
1	Finish engine optimisation unvalidated	Incorrect areas on issued drawings	Run Compare-FinishCsv.ps1 against a production model before office release	Engineering
2	Installers unsigned	SmartScreen warnings; corporate policy may block outright	Procure an organisational code-signing certificate	Management
3	Certificate trust required per workstation	Manual admin step on every machine; dialog defaults to <i>Do Not Load</i>	Resolved by Risk 2	Management

#	Risk	Impact	Action	Owner
4	No automated test coverage	Regressions detectable only in production	Establish a validation model and expected-output baseline	Engineering
5	Duplicate installs	Users silently run outdated code	Mitigated — start-up detection shipped; IT to clear per-user copies before rollout	IT / Engineering
6	Installers not yet executed	Unknown deployment behaviour	Pilot install on one workstation before wider distribution	Engineering
7	AI tier limits halt development mid-task	Idle paid engineering hours; delayed milestones across the Section 6 pipeline	Approve high-tier allocation — see Section 7	Management

9. Recommended Next Steps

1. **Validate the finish engine.** Diff before/after CSV exports on a real project. This gates everything else — it is the only item whose failure mode reaches a drawing.
2. **Decide on code signing.** A single procurement decision closes Risks 2 and 3 together.
3. **Approve high-tier AI allocation (Claude Max ×5 / ×20).** Section 7. This governs delivery rate for the entire Section 6 pipeline and is the lowest-cost decision on this list.
4. **Pilot install** on one workstation, ideally one already carrying a per-user copy, to exercise the new duplicate detection.
5. **Finalise the parameter mapping list**, unblocking Section 3 and allowing that workstream to be re-baselined.
6. **Schedule Automated-Void for Fitting Casework**, sequencing Stair Void Automation immediately behind it to reuse the infrastructure.
7. **Establish a validation model** with known quantities as a permanent regression baseline. The absence of one is the root cause of Risks 1 and 4.

10. Note on Revision

This report supersedes the capability table of the *DKSI Automation Programme — Executive Status Report* (6 August 2026) for the add-in suite specifically. Parameter Linking, previously reported at 100%, is withdrawn to Section 3 at the programme owner's direction pending mapping finalisation. Programme-level items outside the add-in — Scan-to-BIM, QA/QC tier alignment — are unchanged and remain governed by the earlier report.